

Architectures for Embedded Systems

An Interactive Arcade Gaming Console on the ESP32-C6

Eduardo Alves (104179) - eduardoalves@ua.pt

Overview

This work presents ESP-Arcade, a self-contained retro gaming console implemented on an ESP32-C6. The system runs three classic mini-games: Flappy, Pong and Dino Run on an ST7735 TFT display, controlled through a single push button and a potentiometer, with an LED that pulses on every score increment for immediate physical feedback. A DHT20 sensor provides ambient temperature and humidity telemetry, and a microSD card persists high scores across power cycles. The firmware is structured as cooperating FreeRTOS tasks that share a mutex-protected `arcade_state_t` object: a game task drives the menu state machine and per-game physics, a display task renders the screen using a dirty-rectangle strategy to eliminate flicker, a sensor task polls the DHT20, and a network task publishes telemetry. Button presses are delivered from a GPIO interrupt through a shared queue that also accepts synthetic remote commands, unifying local and remote input so that a single button, combined with a 5-second hold gesture, can operate the entire console. The TFT and microSD card share one SPI2 bus through independent chip-select lines. When idle, the device enters Light Sleep to save power and is woken by the same button. Telemetry and remote commands are exchanged with a Node.js web dashboard over MQTT with TLS, using a locally generated certificate authority, giving the console a live online leaderboard and remote control. The result is an integrated embedded application that demonstrates peripheral bus sharing, real-time concurrency, synchronization, non-volatile persistence, graphics optimization, and power management.

Universidade de Aveiro
2026



universidade
de aveiro

1 Functional Requirements and Use Cases

1.1 Use Cases

This system was imagined with several real-world deployment scenarios in mind:

- **Portable / Home Entertainment:** As a compact, self-contained handheld console, it offers a quick and casual gaming break without depending on a smartphone or a computer, and being USB powered lets it be used anywhere a 5 V source is available (power bank, wall charger, laptop port).
- **Shared / Multi-Console Leaderboard:** Since the leaderboard lives on a central MQTT broker rather than on the device itself, several independent consoles could publish to the same broker and compete on one shared leaderboard, a scenario suited to a small arcade corner or a class competition.
- **Complementing Other IoT Setups:** The status and command topics use a simple, self-describing JSON format over standard MQTT, so the console can be absorbed into a larger home-automation or IoT dashboard as one more device, rather than requiring its own dedicated interface.

1.2 Functional Requirements

The system should allow the player to select and play one of three independent games using a single push button and a potentiometer, rendering the active game on the TFT display in real time. It must persist the high score of each game across power cycles on a microSD card, and give ambient temperature and humidity feedback on the menu screen through an I2C sensor without affecting gameplay. To save power, it should automatically enter a low-power sleep state after a period of inactivity and resume immediately on a button press. Finally, after joining an existing Wi-Fi network as a station, it must publish its live state (active screen, current score, high scores) and accept remote commands over an encrypted connection, so that a web dashboard hosted on a companion PC and reachable from any device on the same network can display the leaderboard in real time and control the console remotely.

2 Logical Circuit and Implementation Approach

The following figure shows the resulting wiring: the ESP32-C6 at the centre, the push button, LED and potentiometer on the 3V3 rail to the left, the DHT20 on I2C, and the ST7735 display with its microSD slot sharing one SPI bus, powered independently from 5V, to the right. With the physical connections established, the following section describes how this hardware is organised into a logical, concurrent software architecture.

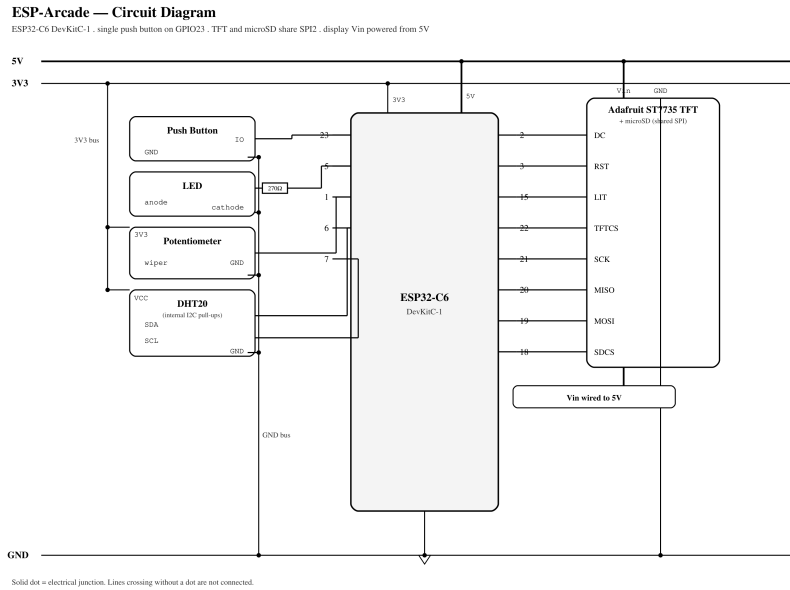


Figura 1: Final State Project Diagram

3 Hardware Implementation

The following photo presents the final breadboard implementation. To improve the clarity of the circuit during visual inspection, different colored jumper wires were used to identify the purpose of each connection, making the implementation easier to understand for first-time viewers.

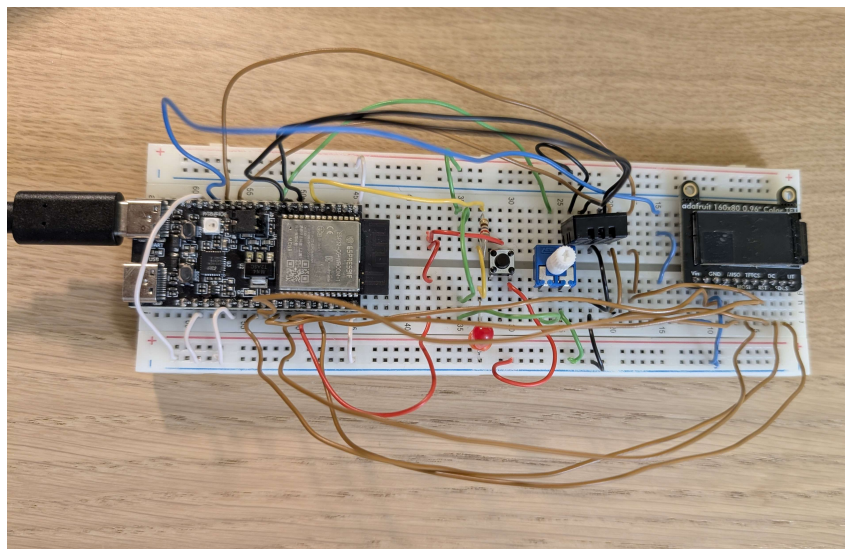


Figura 2: Final breadboard implementation

As we can analyze on the photo above, the **white jumpers** provided the power supply for the ESP32-C6 chip, connecting the ground to the negative rail and 3V3 to the positive rail. The **brown jumpers** were used to make most of the proper connections between the ESP32-C6 and the ADAFRUIT Display/Card Reader. The **red jumpers** were used to make the proper connection of the push Button. The **yellow jumpers** were

used to properly connect the LED light. The **black jumpers** were used for the proper connections of the DHT20. The **green jumpers** were used for the proper connection of the potentiometer. The blue jumpers were used for the visual screening of the ADAFRUIT Display/Card Reader.

- **ESP32-C6:** The ESP32-C6 microcontroller served as the central component of the project, with the entire system being designed and implemented around its capabilities.
- **ADAFRUIT Display/Card Reader:** A combined module providing a 160×80 ST7735 TFT display and a microSD card slot on a single board, used to render the game graphics and the menu, and to host the high-score storage, both sharing the microcontroller’s SPI bus through independent chip-select lines.
- **MicroSD:** A microSD card mounted through the shared SPI bus and managed with the FatFs file system, used to persist the high score of each game so that records survive power loss and are reloaded automatically at boot.
- **Button:** A single push button connected to a GPIO configured with an internal pull-up and a falling-edge interrupt, used as the sole physical control of the console: a short press performs the context-dependent action (start, flap, jump or retry), while holding it for five seconds returns to the main menu from anywhere.
- **270 Ω Resistor:** A current-limiting resistor placed in series with the status LED to protect both the LED and the GPIO pin from excessive current.
- **LED:** A status LED driven by a digital output that pulses briefly every time the player’s score increases, giving immediate visual feedback synchronized with the game state.
- **Potentiometer:** An analog control read through the ADC, calibrated to compensate for its limited usable voltage range on the breadboard, used to navigate the game selection menu and to control the player’s paddle in Pong.
- **DHT20:** A digital temperature and humidity sensor connected over I2C, used purely for ambient telemetry: its readings are displayed on the menu screen and published to the web dashboard, without influencing gameplay.

4 Software Implementation

- **DHT20 Driver:** Implements I2C master communication with the sensor, triggering a measurement and retrieving the current temperature and humidity every two seconds. Reads are wrapped in a non-aborting function so a missing or disconnected sensor simply hides the readout instead of crashing the firmware, leaving gameplay completely unaffected.
- **Display Driver:** Controls the ST7735 TFT via SPI, rendering the menu, the three games and the ambient readout. Instead of redrawing the full 160 × 80 frame every tick, it uses a dirty-rectangle strategy that erases and repaints only the sprites and text that changed since the previous frame, eliminating flicker on the shared SPI bus.

- **SD Card Driver:** Mounts a FAT filesystem (FatFs) over the same SPI bus as the display and persists the three high scores in a tagged text file, rewriting it whenever a record is beaten or a reset command is received from the dashboard, so records survive power loss.
- **MQTT/TLS Client:** Publishes a JSON status payload (active screen, live score, high scores, ambient readout) to `arcade/status` and subscribes to `arcade/command` on a Mosquitto broker. A certificate authority generated locally and compiled into the firmware authenticates the connection, so all traffic is encrypted; the same channel feeds a Node.js web dashboard that shows a live leaderboard and can inject remote commands indistinguishable from a physical button press.

4.1 Input Handling and Power Management

The console is designed to be operable with a single physical button, combining two complementary mechanisms on the same GPIO. Short presses are captured by a falling-edge interrupt that pushes the pin number into a FreeRTOS queue, so the ISR itself does no work beyond that; the game task drains and debounces this queue every tick and reacts according to the current screen (start, flap, jump or retry). In parallel, the same task polls the pin level directly: holding it low for five seconds is detected as a distinct *hold* gesture that returns to the main menu from anywhere, tolerating brief contact drops shorter than 300 ms so a breadboard-grade switch remains reliable.

After 30 seconds of inactivity on the menu, the device enters *Light Sleep*: the backlight pin is frozen with `gpio_hold_en()` so the panel does not flicker during the transition, the button is armed as the wake-up source with `gpio_wakeup_enable()`, and the CPU halts while RAM is retained, unlike Deep Sleep. The button press that wakes the device is discarded from the input queue so it does not accidentally start a game. During normal operation the Wi-Fi radio automatically alternates between active transmission, when the network task publishes telemetry, and modem-sleep while idle between publications, reducing average power draw without any explicit mode selection by the user.

5 Conclusion

ESP-Arcade integrates a shared SPI bus (display and microSD), an I2C ambient sensor, analog and interrupt-driven digital input, persistent storage, light-sleep power management, and authenticated, encrypted telemetry into a single playable, self-contained product. The system is structured as a small set of well-synchronised FreeRTOS tasks around one shared, mutex-protected state, and is fully operable with a single push button through the combination of edge-triggered taps and a polled hold gesture. The result demonstrates that a rich, connected embedded application can be built from a handful of clearly separated concerns input, rendering, storage, sensing and networking.

6 Use of Artificial Intelligence

Artificial Intelligence was used as a tool to refine this report and assist with code development, optimization, and improving overall efficiency of the project. Allowing me as a solo student working on this project to overcome obstacles faster, as well as being a fast source of information regarding the necessary measures to overcome difficulties alone.